

Phase 1 Mini-Project — Full EDA on a Real Dataset

Today is the most important day of Phase 1. You take everything you have learned — Python, NumPy, Pandas, Matplotlib, Seaborn, statistics, and probability — and apply it all to one real dataset. The output is your first proper portfolio piece: a clean, professional, narrative-driven EDA notebook pushed to GitHub with a README. This is what actually gets you interviews.

This is NOT a lecture day. The trainer floats and helps. You build.

Today's deliverable: A complete, well-documented EDA notebook (.py file or .ipynb) pushed to GitHub by end of the day with a proper README.

Time: 90 minutes in class + evening polishing at home.

Assessment: This counts as Portfolio Piece 2 (Day 1 GitHub README was piece 1).

SECTION 01 What is EDA and Why Does it Matter?

Exploratory Data Analysis (EDA) is the process of exploring a dataset to understand it before doing any modelling. It is not glamorous work. It is not building neural networks. But it is the step that separates good data scientists from bad ones.

A famous quote in data science:

'In data science, 80% of time is spent preparing data and 20% modelling.
But 80% of results come from the 20% of modelling.
The preparation is what makes the 20% worth anything.'

EDA is that preparation. You cannot build a good model on data you do not understand.

EDA answers these questions before you build any model:

- What does this dataset actually contain? How many rows? How many columns?
- What data type is each column? Are they stored correctly?
- Are there missing values? How many? Which columns? What should I do about them?
- Are there duplicates or obvious data quality issues?
- What is the distribution of each numeric variable? Skewed? Normal? Uniform?

- Are there outliers? Are they real data or errors?
- Which features are correlated with each other? With the target?
- What patterns, trends, or anomalies exist in the data?
- What questions does this data answer? What questions can I NOT answer from it?

The output of EDA is not just charts and statistics.

The output is a STORY about the data written in clear English.

Every chart must answer a specific question.

Every statistic must be interpreted, not just printed.

Every finding must connect to the business context of the dataset.

A notebook full of charts with no commentary is not EDA.

It is just visualisation. Add the narrative and it becomes analysis.

SECTION 02 Choose Your Dataset

Pick ONE dataset from the options below. Choose something you are genuinely curious about. The best EDA comes from authentic interest in the data.

Option A — Airbnb Listings (Recommended for most students)

A dataset of Airbnb listings from a major city (Mumbai, Bengaluru, or any city you choose).

Contains: listing name, neighbourhood, room type, price, minimum nights, number of reviews, availability, host information.

- Why it is good: relatable, has both numeric and text data, clear business context
- Business question: What factors determine the price of an Airbnb listing?
- Download: [kaggle.com](https://www.kaggle.com), search 'Airbnb listings India' or any city

Option B — IMDb Movies Dataset

A dataset of thousands of movies with: title, genre, year, rating, votes, director, runtime, gross revenue.

- Why it is good: non-tech students relate to movies, interesting patterns
- Business question: What makes a movie successful? High rating or high revenue?
- Download: [kaggle.com](https://www.kaggle.com), search 'IMDb movies dataset'

Option C — NYC Taxi Trips

A dataset of millions of taxi rides in New York with: pickup/dropoff location, trip distance, duration, fare amount, tip amount, payment type.

- Why it is good: great for time patterns, distance vs price relationships
- Business question: What determines how much a passenger tips the driver?
- Download: kaggle.com, use a sample of 50,000 rows for manageability

Option D — Indian Census / Population Data

District-level demographic data for India: population, literacy rate, sex ratio, household size, state, urban vs rural.

- Why it is good: meaningful, impactful, connects to real India
- Business question: What factors correlate with literacy rate across districts?
- Download: kaggle.com or data.gov.in

Option E — Your OWN dataset from Day 7 homework

If you downloaded a Kaggle dataset for Day 7's Exercise 5 that genuinely interests you, use that. Familiarity with the data is an advantage.

How to download from Kaggle:

1. Go to kaggle.com and create a free account if you have not already
2. Search for your chosen dataset
3. Click Download — you will get a ZIP file with one or more CSV files
4. Extract the ZIP and place the CSV in your Day_12 project folder
5. If the file is very large (>100k rows), load only the first 50,000 using:
`df = pd.read_csv('file.csv', nrows=50000)`

SECTION 03 Project Structure — How to Organise Your Work

Professional data science projects have a clean folder structure. Create this structure before writing a single line of code.

□ Recommended folder structure for your project:

```
Day_12/
├── data/
│   └── airbnb_listings.csv    <- raw dataset goes here
├── charts/                  <- all saved PNG files go here
```

```
|— eda.py          <- your main EDA script
|— README.md      <- written last, explains everything
```

How to create this in terminal:

```
mkdir Day_12
cd Day_12
mkdir data charts
touch eda.py README.md
```

Put your CSV file inside the data/ folder.
All plt.savefig() calls should save to the charts/ folder.
Push the entire Day_12 folder to your GitHub repo.

SECTION 04 The EDA Pipeline — 7 Steps to Follow in Order

Follow these seven steps for every EDA you ever do. They are not optional. This sequence is the professional standard.

- 1 Load and first look**
Read the CSV, print shape, head, tail. See what you are working with before doing anything else.
- 2 Understand the structure**
Print column names, data types (dtypes), and df.info(). Identify which columns are numeric, which are categorical, which are dates.
- 3 Assess data quality**
Count missing values per column (percentage). Identify duplicates. Check for impossible values (negative ages, prices of zero, future dates in past data).
- 4 Clean the data**
Handle missing values (drop or fill). Remove duplicates. Fix data types. Standardise text columns. Save the cleaned version.
- 5 Univariate analysis**
Analyse each column individually. Histograms for numeric columns. Count plots / value_counts for categorical columns. Describe distributions in words.
- 6 Bivariate and multivariate analysis**

Explore relationships between columns. Scatter plots, box plots, correlation heatmap, groupby summaries. Ask: which features relate to the target?

7

Summarise insights and next steps

Write 5-10 key findings in plain English. State what ML model you would build next. State what additional data would help. This section is what separates analysts from coders.

SECTION 05 Complete EDA Template — Adapt This for Your Dataset

This is a complete, production-quality EDA template. Every section is labelled with what to write and why. Adapt it to your chosen dataset. Do NOT just run it as-is on the Titanic — that is not the point. The point is to understand each step well enough to apply it to new data.

Step 1 — Setup and Load

```
# =====
# EDA: [Your Dataset Name]
# Author: [Your Name]
# Date: [Today's Date]
# Business Question: [Write your specific question here]
# =====

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings

warnings.filterwarnings('ignore')    # Suppress minor warnings
sns.set_theme(style='whitegrid')     # Clean chart style
plt.rcParams['figure.figsize'] = (10, 6) # Default figure size

# — Load Data —————
df = pd.read_csv('data/your_dataset.csv') # Update filename

# First look
print('=' * 60)
print('DATASET: [Your Dataset Name]')
print('=' * 60)
print(f'Shape: {df.shape[0]:,} rows x {df.shape[1]} columns')
print(f'Memory usage: {df.memory_usage(deep=True).sum() / 1024**2:.2f} MB')
print()
print('First 5 rows:')
print(df.head())
print()
print('Last 5 rows:')
```

```
print(df.tail())
```

Output:

```
=====
DATASET: Airbnb Mumbai Listings
=====
Shape: 18,524 rows x 16 columns
Memory usage: 2.37 MB

First 5 rows:
      id                                name  host_id  neighbourhood
room_type  price
0    1  Cozy room near Bandra station    1234  Bandra West  Private room
2500
...
```

Step 2 — Understand the Structure

```
# — Column types and info —————
print('\nColumn Information:')
print('=' * 60)
df.info()

print('\nColumn Names:')
for i, col in enumerate(df.columns, 1):
    print(f' {i:2d}. {col} ({df[col].dtype})')

# Separate numeric and categorical columns
numeric_cols = df.select_dtypes(include='number').columns.tolist()
categorical_cols = df.select_dtypes(include='object').columns.tolist()

print(f'\nNumeric columns ({len(numeric_cols)}): {numeric_cols}')
print(f'Categorical columns ({len(categorical_cols)}): {categorical_cols}')

# Statistical summary of numeric columns
print('\nNumeric Summary:')
print(df[numeric_cols].describe().round(2))
```

Step 3 — Data Quality Assessment

```
# — Missing values —————
print('\nMISSING VALUE REPORT')
print('=' * 50)
missing = pd.DataFrame({
    'count' : df.isnull().sum(),
    'percentage' : (df.isnull().sum() / len(df) * 100).round(1)
})
missing = missing[missing['count'] > 0].sort_values('percentage',
ascending=False)
```

```

if len(missing) == 0:
    print(' No missing values found!')
else:
    print(missing)
    print(f'\n Total missing values: {df.isnull().sum().sum():,}')

# — Duplicates —————
n_dupes = df.duplicated().sum()
print(f'\nDuplicate rows: {n_dupes} ({n_dupes/len(df)*100:.1f}%)')

# — Unique values per categorical column —————
print('\nUnique values per categorical column:')
for col in categorical_cols:
    n_unique = df[col].nunique()
    print(f' {col}: {n_unique} unique values')
    if n_unique <= 10:
        print(f' Values: {df[col].value_counts().index.tolist()}')

# — Sanity checks on numeric columns —————
print('\nSanity checks (min values of numeric columns):')
print(df[numeric_cols].min())
# Look for: negative prices, zero quantities, impossible values

```

Step 4 — Clean the Data

```

# — Clean a copy, always keep original —————
df_clean = df.copy()
original_shape = df_clean.shape

# 1. Drop columns that are >60% missing or not useful
# (Adapt these column names to YOUR dataset)
cols_to_drop = [] # Add column names here after inspecting
if cols_to_drop:
    df_clean = df_clean.drop(columns=cols_to_drop)
    print(f'Dropped columns: {cols_to_drop}')

# 2. Handle missing values
# For each numeric column with missing values:
for col in numeric_cols:
    if df_clean[col].isnull().sum() > 0:
        fill_val = df_clean[col].median() # Use median for skewed data
        df_clean[col] = df_clean[col].fillna(fill_val)
        print(f' Filled {col} missing values with median: {fill_val:.2f}')

# For categorical columns:
for col in categorical_cols:
    if df_clean[col].isnull().sum() > 0:
        fill_val = df_clean[col].mode()[0]
        df_clean[col] = df_clean[col].fillna(fill_val)
        print(f' Filled {col} missing values with mode: {fill_val}')

```

```

# 3. Remove duplicates
df_clean = df_clean.drop_duplicates()

# 4. Summary
print(f'\nCleaning Summary:')
print(f'   Before: {original_shape}')
print(f'   After  : {df_clean.shape}')
print(f'   Missing values remaining: {df_clean.isnull().sum().sum()}')

# 5. Save cleaned version
df_clean.to_csv('data/cleaned.csv', index=False)
print('   Cleaned dataset saved.')

```

Step 5 — Univariate Analysis

Analyse each column individually. For every chart, write ONE sentence of insight as a comment or print statement. No silent charts.

```

# — Numeric columns: Histograms —————
n_numeric = len(numeric_cols)
n_cols    = 2
n_rows    = (n_numeric + 1) // 2

fig, axes = plt.subplots(n_rows, n_cols,
                          figsize=(14, 4 * n_rows))
axes = axes.flatten()

for i, col in enumerate(numeric_cols):
    axes[i].hist(df_clean[col].dropna(), bins=30,
                 color='#2196F3', edgecolor='white', linewidth=0.8)
    axes[i].axvline(df_clean[col].median(), color='red',
                    ls='--', lw=2, label=f'Median:
{df_clean[col].median():.1f}')
    axes[i].set_title(f'Distribution of {col}', fontweight='bold')
    axes[i].set_xlabel(col)
    axes[i].set_ylabel('Count')
    axes[i].legend(fontsize=10)
    axes[i].grid(axis='y', alpha=0.5)

# Hide empty subplots
for j in range(i+1, len(axes)):
    axes[j].set_visible(False)

plt.suptitle('Distribution of All Numeric Variables',
             fontsize=16, fontweight='bold', y=1.01)
plt.tight_layout()
plt.savefig('charts/01_univariate_numeric.png', dpi=150,
            bbox_inches='tight')
plt.show()
print('\nChart saved: 01_univariate_numeric.png')

```

```

# — Categorical columns: Count plots —————
for col in categorical_cols:
    if df_clean[col].nunique() <= 15: # Only plot if manageable categories
        plt.figure(figsize=(10, 5))
        vc = df_clean[col].value_counts().head(10)
        bars = plt.bar(vc.index, vc.values, color='#4CAF50',
                        edgecolor='white')

        for bar in bars:
            plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() +
0.5,
                    str(int(bar.get_height())), ha='center', fontsize=10,
fontweight='bold')

        plt.title(f'Top 10 Values in: {col}', fontsize=14,
fontweight='bold')
        plt.xlabel(col); plt.ylabel('Count')
        plt.xticks(rotation=45, ha='right')
        plt.tight_layout()
        plt.savefig(f'charts/02_univariate_{col}.png', dpi=150,
bbox_inches='tight')
        plt.show()

```

Step 6 — Bivariate and Multivariate Analysis

This is where the real insights live. You are looking for RELATIONSHIPS between columns, especially with the target variable.

```

# — Correlation Heatmap —————
plt.figure(figsize=(12, 8))
corr_matrix = df_clean[numeric_cols].corr()

sns.heatmap(
    corr_matrix,
    annot      = True,
    fmt        = '.2f',
    cmap       = 'coolwarm',
    center     = 0,
    square     = True,
    linewidths = 0.5,
    cbar_kws   = {'shrink': 0.8}
)
plt.title('Correlation Matrix', fontsize=15, fontweight='bold', pad=15)
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.savefig('charts/03_correlation_heatmap.png', dpi=150,
bbox_inches='tight')
plt.show()

# Print top correlations with target variable
# (Replace 'price' with YOUR target column name)

```

```

target_col = 'price' # <-- CHANGE THIS to your target column
if target_col in numeric_cols:
    print(f'\nTop correlations with {target_col}:')
    corr_with_target = corr_matrix[target_col].drop(target_col)
    print(corr_with_target.sort_values(key=abs, ascending=False))

# — Box plots: numeric by category —————
# Compare distribution of target across categories
# (Replace column names to match YOUR dataset)
for cat_col in categorical_cols[:3]: # First 3 categorical columns
    if df_clean[cat_col].nunique() <= 8:
        plt.figure(figsize=(12, 6))
        sns.boxplot(
            data = df_clean,
            x = cat_col,
            y = target_col,
            palette = 'Blues'
        )
        plt.title(f'{target_col} by {cat_col}',
                  fontsize=14, fontweight='bold')
        plt.xticks(rotation=30, ha='right')
        plt.tight_layout()
        plt.savefig(f'charts/04_box_{cat_col}.png', dpi=150,
bbox_inches='tight')
        plt.show()

# — Scatter plots: key numeric relationships —————
# Pick the 2-3 numeric features most correlated with target
if target_col in numeric_cols:
    top_features = corr_matrix[target_col].drop(target_col)\
        .sort_values(key=abs, ascending=False).head(2).index

    fig, axes = plt.subplots(1, len(top_features), figsize=(14, 5))
    if len(top_features) == 1:
        axes = [axes]

    for ax, feat in zip(axes, top_features):
        ax.scatter(df_clean[feat], df_clean[target_col],
                  alpha=0.4, color='#2196F3', s=20)
        ax.set_xlabel(feat, fontsize=12)
        ax.set_ylabel(target_col, fontsize=12)
        ax.set_title(f'{feat} vs {target_col}', fontweight='bold')
        ax.grid(True, alpha=0.4)

    plt.suptitle('Key Feature Relationships', fontsize=15,
fontweight='bold')
    plt.tight_layout()
    plt.savefig('charts/05_scatter_relationships.png', dpi=150,
bbox_inches='tight')
    plt.show()

```

Step 7 — Insights Summary (The Most Important Step)

This section is what turns data analysis into data storytelling. Write this in plain English that a non-technical manager could read and understand.

```
# — Key Findings Summary —————
print()
print('=' * 60)
print('KEY FINDINGS FROM EDA')
print('=' * 60)

# Write YOUR actual findings here based on what you observed.
# Template structure to follow:

findings = [
    '1. DATASET: [dataset name] has [X] rows and [Y] columns after',
    'cleaning.',
    '    [X]% of data was missing in [column name], handled by [method].',
    '',
    '2. DISTRIBUTION: [numeric column] is [right/left/normally]',
    'distributed.',
    '    The median [value] differs from mean [value], suggesting',
    '[outliers/skew].',
    '',
    '3. STRONGEST RELATIONSHIP: [feature A] and [target] have correlation',
    'of [X].',
    '    This means [plain-English interpretation].',
    '',
    '4. CATEGORY INSIGHT: [category] has significantly [higher/lower]',
    '[target].',
    '    Specifically [Group A] averages [X] while [Group B] averages [Y].',
    '',
    '5. OUTLIERS: [X] data points were identified as outliers in',
    '[column].',
    '    These represent [interpretation of what outliers mean in this',
    'context].',
    '',
    '6. BUSINESS IMPLICATION: Based on this EDA, the most important',
    'features',
    '    for predicting [target] appear to be [feature1, feature2,',
    'feature3].',
    '',
    '7. NEXT STEPS: An ML model (suggest type) could be built to predict',
    '[target].',
    '    Additional data on [what] would strengthen the analysis.',
]

for line in findings:
    print(line)
```

NEVER do this:

- Print a statistic without interpreting it
- Show a chart without explaining what it means
- Write 'the mean is 45.3' without saying what 45.3 means in context

ALWAYS do this:

- 'The mean listing price is Rs. 3,200 per night, but the median is Rs. 1,800, suggesting a right-skewed distribution pulled up by luxury properties.'
- 'Listings in Bandra cost on average 2.4x more than listings in Kurla, confirming the neighbourhood is the strongest price predictor.'

SECTION 06 Writing a Great README — Your Project's Cover Letter

The README is the first thing a recruiter or interviewer sees when they click your GitHub repo link. A bad README makes even great code look unprofessional. A great README makes even average code look impressive. Write it last, after all your analysis is done.

README template — copy and adapt this exactly

```
# [Your Dataset Name] — Exploratory Data Analysis

## Overview
A complete EDA on [dataset name] answering the question:
**[Your specific business question]**

**Dataset:** [Name] | [Source: Kaggle/data.gov.in] | [X] rows x [Y] columns
**Tools used:** Python, Pandas, NumPy, Matplotlib, Seaborn

---

## Key Findings

1. **[Finding 1 headline]**
   [One sentence explaining what you found and why it matters]

2. **[Finding 2 headline]**
   [One sentence explanation]

3. **[Finding 3 headline]**
   [One sentence explanation]

4. **[Finding 4 headline]**
   [One sentence explanation]

5. **[Finding 5 headline]**
   [One sentence explanation]

---
```

```

## Charts

### Distribution of Key Variables
![Distributions] (charts/01_univariate_numeric.png)

### Correlation Heatmap
![Correlation] (charts/03_correlation_heatmap.png)

### [Most Interesting Chart Title]
![Key Insight] (charts/04_box_[column].png)

---

## Data Cleaning Summary
- Original dataset: [X] rows x [Y] columns
- Removed [N] duplicate rows
- Filled missing values in [columns] using [method]
- Dropped [columns] due to >60% missing values
- Final clean dataset: [X] rows x [Y] columns

---

## How to Run
```
git clone [your repo URL]
cd Day_12
pip install pandas numpy matplotlib seaborn
python3 eda.py
```

## Next Steps
Based on this EDA, the most promising ML model would be [model type]
predicting [target variable] using [top features].

---
*EDA completed as part of Infiway Global AI/ML Master Program – Phase 1*

```

README pro tips:

Embed your charts directly in the README using the

![Alt text](path/to/image.png) syntax.

GitHub renders them inline — the recruiter sees your charts without even opening your code.

Write your key findings as headlines, not sentences.

'Bandra listings cost 2.4x more than Kurla listings' is a headline.

'I found that the prices are different in different areas' is not.

Keep the README to one scrollable page. If it is too long, trim it.

SECTION 07 Pushing to GitHub — Making It Public

Your code means nothing if it only lives on your laptop. Push it to GitHub so the world (and recruiters) can see it. Follow these exact steps.

□ Pushing your project to GitHub

Step 1: Initialise Git in your Day_12 folder (if not already done)

```
cd Day_12  
git init
```

Step 2: Add a .gitignore file to avoid pushing large data files

Create a file called .gitignore with this content:

```
data/*.csv  
__pycache__/  
*.pyc  
.DS_Store
```

Step 3: Stage all files

```
git add .
```

Step 4: Commit with a meaningful message

```
git commit -m 'Add Phase 1 EDA: Airbnb Mumbai listings analysis'
```

Step 5: Push to GitHub

```
git push origin main
```

Step 6: Go to your GitHub repo, click the repo name, pin this repo

Profile -> Customize your pins -> Select this repo

Note: Add your CSV to .gitignore if it is larger than 25 MB.

GitHub has a 25 MB file limit. Large datasets: add a 'Data' section in README with a download link to the Kaggle page.

SECTION 08 The EDA Rubric — How Your Work is Assessed

Your EDA submission will be assessed on these criteria. Read them before you start so you build the right things.

Criterion	Weight	What to do
Problem framing	10%	Is there a clear business question? Is it stated at the top?
Data quality assessment	20%	Are missing values found and handled correctly? Duplicates checked?
Visualisations	25%	At least 6 charts. All have titles, axis labels, and grids.
Insight narrative	30%	Are findings written in plain English? Do they answer the question?
Code quality	10%	Is code readable? Functions used? Comments added?
README and GitHub	5%	Is README complete? Charts embedded? Repo pinned?

The most common mistakes that lose marks:

1. Charts without titles or axis labels — always add them, no exceptions
2. Printing statistics without interpreting them
3. Using the Titanic dataset instead of a new one
4. README with just 'This is my EDA project' and nothing else
5. Only univariate analysis — the best insights come from relationships between columns
6. Forgetting to save charts as PNG files for the README

SECTION 09 Phase 1 Completion Checklist

Before you call your Day 12 project complete, tick every box on this list. If any box is unticked, finish it tonight.

Phase 1 — Technical Skills

- ☐ Python basics: variables, data types, operators, input/output
- ☐ Data structures: lists, tuples, dictionaries, sets
- ☐ Control flow: if/elif/else, for loops, while loops, break, continue
- ☐ Functions: define, call, parameters, return values, default args, docstrings

- Modules: import math, random, datetime. Create your own module.
- NumPy: arrays, indexing, slicing, broadcasting, statistical operations
- Pandas: Series, DataFrame, read_csv, head/info/describe, loc/iloc, filtering
- Data cleaning: missing values, duplicates, data types, string cleaning, merge, groupby
- Visualisation: line, bar, histogram, scatter, pie, boxplot, violin, heatmap, pair plot
- Statistics: mean/median/std, z-scores, distributions, correlation
- Probability: conditional probability, Bayes theorem, hypothesis testing
- Math: slope, cost function, derivatives, gradient descent, learning rate

Phase 1 — Portfolio

- GitHub account created with a professional username
- Daily homework files (day1 to day11) pushed to GitHub repo
- Day 12 EDA project in its own folder with clean structure
- README written with key findings and embedded charts
- Day 12 repo pinned on your GitHub profile

Phase 1 — Interview Readiness

- Can explain AI vs ML vs DL in 2 minutes without notes
- Can explain what a DataFrame is and demonstrate loc vs iloc
- Can explain the bias-variance trade-off with an example
- Can explain gradient descent in plain English
- Can walk through your Day 12 EDA project end to end

SECTION 10 What Comes Next — Phase 2 Preview

You have completed 12 days of building the foundation. Phase 2 begins on Day 13 and it is where machine learning actually starts. Here is exactly what is coming.

Day	Topic	What you will build
Day 13	ML mental model	Your first real sklearn model, train-test split
Day 14	Linear Regression	Predict house prices, compute MSE and R-squared
Day 15	Ridge, Lasso, ElasticNet	Handle overfitting in regression automatically
Day 16	Logistic Regression	Predict yes/no outcomes, compute precision/recall/AUC
Day 17	Decision Trees	Visualise a model that makes decisions like a flowchart
Day 18	Random Forest	Build an ensemble of 100 trees, read feature importances
Day 19	XGBoost & LightGBM	The models that win Kaggle competitions
Day 20	KNN, Naive Bayes, SVM	Three different approaches to classification
Day 21-23	Feature engineering	Make your data 10x better before modelling
Day 26-28	Mini-project 1	Deploy a complete ML app on Streamlit Cloud

Important: Phase 2 uses EVERYTHING from Phase 1.

Every sklearn model internally uses NumPy arrays (Day 5-6).

Every model needs clean data (Days 7-8).

Every model needs evaluation using statistics (Day 10).

Every model trains using gradient descent (Day 11).

Your EDA skills (Day 12) are used before building EVERY model.

If anything from Days 1-12 feels shaky, revisit it this weekend.

Phase 2 moves fast and assumes Phase 1 is solid.

SECTION 11 Today's Class Schedule

Day 12 is structured differently from all other days. Here is what happens during class time.

Time	What happens
0-10 min	Project brief and rubric walkthrough. Dataset selection finalised for each student.
10-20 min	Project setup: folder structure created, CSV downloaded, first <code>df.head()</code> printed.

20-55 min	Build session. Trainer and Tier-A buddies float and help. Steps 1-4 completed.
55-75 min	Continue build. Steps 5-6 completed. At least 4 charts saved to charts/ folder.
75-85 min	Three students share screen and walk through one insight from their EDA.
85-90 min	Homework brief. Step 7 (insights) and README to be finished tonight.

SECTION 12 Homework — Final Submission for Day 12

This is your Phase 1 capstone. It goes on your GitHub profile and will be shown to every interviewer. Put in the extra hour tonight to make it excellent. A polished Day 12 project has literally gotten students interview calls.

Tonight's tasks — must complete before tomorrow's class

1. Finish all 7 steps of the EDA pipeline in your eda.py file.
2. Make sure every chart has: a descriptive title, labelled axes, a grid, and is saved as a PNG in the charts/ folder.
3. Write the insights summary (Step 7) with at least 5 specific, interpreted findings. No 'the mean is X' without saying what X means.
4. Write the README.md using the template from Section 06. Embed at least 3 chart images in the README.
5. Push everything to GitHub. Confirm the README renders correctly on github.com by viewing the repo in your browser.
6. Pin the Day_12 repo on your GitHub profile: Profile -> Customize your pins -> select the repo.
7. Submit your GitHub repo link to the trainer. This is your Portfolio Piece 2.

Presentation preparation — 5 minutes tomorrow

Three students will present their EDA tomorrow at the start of class. If the trainer picks you, be ready to answer:

- What dataset did you choose and why?
- What was the most interesting finding you discovered?
- If you were to build an ML model on this data, which features would you use and why?

- What data quality issues did you find and how did you handle them?
- What question does this data NOT answer that you wish it could?

Gate 1 Assessment — End of Day 12

Day 12 is Gate 1 of the program. The trainer will assess whether you are ready to move to Phase 2. The question is simple:

Gate 1 Question: Can this student independently load, clean, visualise, and summarise a CSV dataset without any help?

If YES — move to Phase 2 with the cohort starting Day 13.

If NO — intensive remediation plan: buddy pairing, extra office hours, and a revised version of the EDA submitted by Day 14.
Phase 2 participation continues but with closer monitoring.

This is not a punishment. It is early intervention.
A student who struggles with Pandas now will drown in scikit-learn.
Better to identify and address it here than in Phase 3.

Phase 1 Complete. Phase 2 Begins Tomorrow.

You have come a long way in 12 days.

Day 1: You set up Python and pushed a README to GitHub.

Day 12: You independently explored a real-world dataset, cleaned it, built 6+ charts, extracted meaningful insights, and produced a professional portfolio piece that you own forever.

That is not a small thing. Most people who want to 'learn data science' never get this far. You did.

Tomorrow, Day 13, the machine learning begins.
Phase 2 is where all the math from Days 10-11 starts making models.
It is the part everyone was waiting for. Come ready.